

CMSE 801 - Day 21



Defining and using Python classes (continued)

November 13, 2024

General Course Announcements

- Homework 4 is due this **Friday, November 15, at 11:59pm**.
- Quiz 3 and Homework 3 grades are in D2L.
- Project Draft Presentations will be during class on **Wednesday, November 20** (Day 23).
 - Details are on the [course website](#).

Questions/comments from Day 21 PCA

- Lots of good explanations of inheritance versus composition, but still an uncertainty about exact how they are different.
- Some general comments about getting familiar with the new syntax.
 - What's up with `super().__init__()`? **This is part of *inheritance*.**
We'll talk about this.
- Some struggles with the last part of the assignment and using objects inside of other objects (**composition**).
 - And also involved dictionaries!

What is "inheritance" and why would we use it?

Inheritance increases the **reusability** of our code and creates space for a hierarchy within classes (objects) we build.

Imagine that we have some sort of common "base" class that contains a set of attributes and methods that all versions of that class should have, but then we realize that, for some use cases, it would be useful to have a few additional methods or attributes.

We can use inheritance to create a "derived" class which is guaranteed to have all of the same attributes and methods of the base class, but with some added functionality that only makes sense for the new class.

The `super().__init__()` code is what ensures that our derived classes are initialized in the same way that the base class is initialized.

Let's briefly revisit how this was done when created new "Animals"

```
In [ ]: class Animal:
        """
        This is an Animal class, used to store information about animals
        Or for characters in the Madagascar movies.
        """
        def __init__(self, kind, name, to_zoo='', area=100, staff=1):
            """ Initialize the Animal object.
            Inputs are
            kind : the type of animal (string)
            name : the nickname for the animal (string)
            to_zoo : the date the animal came to the zoo (in 'yyyy-mm-dd' format)
            """
            self.kind = kind
            self.name = name
            self.brought_to_zoo = to_zoo
            self.required_area = area # required area [m2]
            self.required_staff = staff # required number of staff members
```

```
In [ ]: class Penguin(Animal):
        def __init__(self, name):
            """ Initialize Penguin class. All we need now is the animal nickname
            kind = 'penguin'
            # this calls initialization of Animal class and setting the animal attributes
            super().__init__(kind, name)
            # in lines below we define attribute values for penguins
```

```
self.required_area = 300  
self.required_staff = 0.5
```

What is "composition" and why would we use it?

Composition is a design principle where classes are built using instances of other classes, creating a "has-a" relationship instead of the "is-a" relationship typical of inheritance. Instead of subclassing or extending another class, you create a class that includes instances of other classes as fields, allowing you to combine their functionality in a flexible way. This means one object contains (or is made up of) other objects as its parts or components.

The key advantages of composition are that it allows you to:

- Build complex systems from simpler components
- **Reuse** objects in different contexts
- Keep related data and functionality bundled together

It also provide some **flexibility** over inheritance: With inheritance, changes in a base class affect all subclasses, which can lead to tight coupling and hard-to-maintain code. Composition allows objects to be combined in flexible ways, as you can swap out specific parts without impacting the entire structure.

Let's take a look at how we can use `Animal` objects inside the `Zoo` methods.

The extra methods from part 3.3/3.4:

In [1]:

```
def get_animal_names(self, sort=False):  
    """  
    Return animal names in the Zoo.  
    Return sorted names if parameters sorted==True  
    """  
    names = list(self.animals.keys())  
    if sort == True:  
        # we have to return sorted names; so sort the names  
        names = sorted(names)  
  
    return names  
  
def total_area(self):  
    """  
    Return the total Zoo area to host all animals defined in the ob  
    """  
    area = 0  
    for key in self.animals:  
        area += self.animals[key].get_required_area()  
  
    ## or accessing values directly  
    # for animal in self.animals.values():  
    #     area += animal.get_required_area()  
  
    return area
```

Brief aside on Python dictionaries

```
In [3]: # Create a dictionary of animal space requirements
animal_space = {"lion": 100, "zebra": 50, "penguin": 20}

# I can add additional entries as needed
animal_space['monkey'] = 10

# Access a single value using its key
print(animal_space["lion"]) # prints: 100

# Get all keys
print("\nKeys:")
for animal in animal_space.keys():
    print(animal)

# Get all values
print("\nValues:")
for space in animal_space.values():
    print(space)
```

100

Keys:

lion

zebra

penguin

monkey

Values:

100

50

20

10

```
In [4]: from Animals import Animal, Penguin, Zebra, Lion
        from Zoo import Zoo

        alex = Lion('Alex')
        marty = Zebra('Marty')
        skipper = Penguin('Skipper')
        kowalski = Penguin('Kowalski')

        # create a Zoo and adding Alex and Skipper
        zoo = Zoo('NYC ZOO')
        zoo.add(alex)
        zoo.add_animal(skipper)
```

```
In [5]: zoo.animals
```

```
Out[5]: {'Alex': <Animals.Lion at 0x1069af830>,
         'Skipper': <Animals.Penguin at 0x1069f39b0>}
```

```
In [6]: zoo.animals['Alex'].required_area
```

```
Out[6]: 600
```

Goals for today

- Practice writing a new class from scratch (to practice things you learned last class)
- Create objects using both your code and some provided code, some of which uses object **inheritance**.
- Put it all together by combining classes/objects (**composition**)

Looking forward - Day 22 PCA:

Reviewing/revisiting random number generation

- Review/remind yourself how we can generate random numbers in Python
- We'll be using random numbers for modeling and data fitting purpose in the remaining course activities

